AirlineMS - Setup, IDE, and Demo Guide

Windows installation and run instructions for Eclipse, VS Code, MySQL Workbench 8, and Command Prompt

This guide is for setting the project up on another Windows computer. Run the commands from the project root folder (`D:\Airline` in the examples). The application uses local demo credentials and a local root MySQL account; keep it on a classroom machine and do not expose it to the public internet.

1. What must be installed or running?

Tool	Project version	What it is needed for
OpenJDK / Temurin JDK	17.0.12	Compiles and runs all five Spring Boot applications.
Maven	3.9.14	Builds the six-module Maven reactor and runs Java tests.
Node.js	22.9.0	Builds Angular and runs its optional development server.
npm	10.8.3	Installs exact frontend dependencies from `package-lock.json`.
Angular CLI	19.2.27 local	Project commands `npx ng ...`; installed by `npm ci`.
Angular packages	19.2.25	Framework packages in this project's lockfile.
MySQL Server	8.0; tested with 8.0.46	Must be running on port 3306.
MySQL Workbench	8.0, optional	A desktop client for looking at schemas and rows; it is not the database server.
Eclipse IDE / VS Code	Optional	Editors and launchers; either can build/run the same files.

Angular 19.2's historical Node.js compatibility range includes Node 22.9.0, which is why this exact version combination works. Angular 19 is now out of support, however; this version is pinned to match the classroom laptop rather than to recommend a new project use it. The official Angular version compatibility table lists the compatibility ranges and support status.

Option A: use the supplied portable tools

The setup scripts put tools in the ignored `.tools` folder and do not change the machine-wide PATH.

```
# Run from D:\Airline
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\setup-local-tools.ps1
. .\scripts\use-local-tools.ps1
java -version
javac -version
node --version
npm --version
mvn --version
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\check-laptop.ps1
```

`setup-local-tools.ps1` downloads checksum-verified Node, JDK, and Maven archives. `use-local-tools.ps1` sets environment variables and PATH only in the PowerShell window where it is dot-sourced. Open a new PowerShell window and run the dot-source line again when you need the pinned commands there.

Install the project-local Angular packages and check versions:

```
Push-Location airline-frontend
npm ci
npx ng version
Pop-Location
```

The local CLI takes precedence over a different globally installed Angular CLI. Do not use a global CLI to generate or upgrade this project.

Option B: use tools already installed on the computer

If you use machine-installed tools, make sure each terminal is using the matching JDK 17, Maven, Node 22.9, and npm 10.8.3. Check with `where java`, `java -version`, `where node`, `node --version`, `npm --version`, and `mvn --version`. Node 22.9.0 includes npm 10.8.3 for this project. You still need to run `npm ci` in `airline-frontend` to install its local Angular CLI and packages.

2. Start MySQL Server

Use the supplied portable MySQL Server

1. From the project root, run:

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\setup-local-mysql.ps1
```

This downloads and initializes MySQL Server under `.tools`; it does not register a Windows service.

2. In its own PowerShell window, start the server and keep that window open:

```
cd D:\Airline
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\start-local-mysql.ps1
```

3. On the very first start only, set the local classroom root password using the `mysql.exe` command printed by the setup script. This demo uses `password` by default. If another MySQL installation already owns port 3306, use that server instead and set the `DB_PASSWORD` environment variable to its actual password.

Use an existing MySQL Server

Start MySQL Server 8.0 and confirm it listens on `127.0.0.1:3306`. Set user/password access as required by your local MySQL administrator. The sample configuration expects user `root` and password `password`; override the password for each service process with:

```
$env:DB_PASSWORD = 'your-local-mysql-password'
```

In Command Prompt, use `set DB_PASSWORD=your-local-mysql-password` in each service window.

Optional: inspect the database with MySQL Workbench 8.0

1. Open Workbench and create a connection using **Standard TCP/IP**.

2. Set hostname `127.0.0.1`, port `3306`, and the local database username/password.

3. Use **Test Connection**, then connect.

4. Open `database/create-databases.sql` in a SQL editor tab and run it. Or the three service JDBC URLs create the schemas on local startup when the selected user has permission.

5. In the Schemas sidebar, refresh. After starting the services, inspect `airline_auth.users`, `airline_flights.flights`, `airline_bookings.bookings`, and `airline_bookings.seat_allocations`.

Workbench connects to a running MySQL server; it does not replace it. The official Workbench TCP/IP connection guide describes host, port, user, and default schema. The Workbench FAQ describes how to create schemas from the Schemas panel.

3. Create the JWT keys

The RSA key files are intentionally ignored by Git. On a new machine or fresh clone, open a terminal at the project root and run this once:

```
java tools/GenerateKeys.java
```

The Auth service uses `keys/private.pem` to sign JWTs. Auth, Flight, and Booking use the matching `keys/public.pem` to verify signatures. Run the command from the repository root because the generator writes to the relative path `keys/`. Keep

both files together for this local demo; never commit or publicly share the private key. If Auth already has tokens in use, do not regenerate the pair until all services are stopped and clients are ready to sign in again.

`scripts/package-demo.ps1` generates a key pair automatically if `keys/private.pem` is missing. If you run `mvn clean package` directly and then launch services yourself, generate the keys first.

4. Build the project

Presentation build - Angular served by Gateway

This is the simpler mode for a presentation. It bundles the production Angular output into Gateway, so after build the UI is served from port 8080 and does not need `npm start`.

```
# From D:\Airline
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\package-demo.ps1
```

The script loads project-local tools, generates keys if missing, runs `npm ci`, runs the Angular production build, copies its browser assets into Gateway's static folder, and runs `mvn clean package`. The runnable JARs are written to each module's `target` folder. On Windows, if `npm ci` reports `EBUSY` while removing a package folder, stop any Angular dev server and retry once; the clean install has succeeded on retry in this workspace.

Development build - Angular on its own server

```
# From the project root, with pinned tools loaded
. .\scripts\use-local-tools.ps1
java tools/GenerateKeys.java
mvn clean package

Push-Location airline-frontend
npm ci
npm run build
npm start
```

Keep the terminal running `npm start` open. Visit `http://localhost:4200`; `proxy.conf.json` forwards `/api` calls to Gateway at port 8080. Build and start the Java services in separate terminals as described next. For a plain production build without starting the dev server, use `npm run build` instead of `npm start`.

5. Start the five Java processes

Run each command in its **own terminal**, from the repository root. Use the same `DB_PASSWORD` value in each terminal if it is not the default.

1. Eureka:

```
java -Xms64m -Xmx192m -jar discovery-server/target/discovery-server-1.0.0.jar
```

Wait until `http://localhost:8761` opens.

2. Auth:

```
java -Xms64m -Xmx192m -jar auth-service/target/auth-service-1.0.0.jar
```

3. Flight:

```
java -Xms64m -Xmx192m -jar flight-service/target/flight-service-1.0.0.jar
```

4. Booking:

```
java -Xms64m -Xmx256m -jar booking-service/target/booking-service-1.0.0.jar
```

5. In Eureka at `http://localhost:8761`, wait for `AUTH-SERVICE`, `FLIGHT-SERVICE`, and `BOOKING-SERVICE` to show UP. Then start Gateway:

```
java -Xms64m -Xmx192m -jar api-gateway/target/api-gateway-1.0.0.jar
```

The Gateway should start after its routes can be discovered. The demo properties refresh registry/cache data every five seconds to limit startup delay; Spring's Eureka configuration reference documents both properties and their defaults. The five JVM memory flags are upper bounds for Java heaps, not the entire memory use of Windows, MySQL, or the IDE.

Equivalent Command Prompt commands

Open one **Command Prompt** window for each Java process. Example setup for every window:

```
cd /d D:\Airline
set "JAVA_HOME=D:\Airline\.tools\jdk-17.0.12+7"
set "MAVEN_HOME=D:\Airline\.tools\apache-maven-3.9.14"
set "PATH=%JAVA_HOME%\bin;%MAVEN_HOME%\bin;D:\Airline\.tools\node-v22.9.0-win-x64;%PATH%"
```

Then run the same `java -Xms... -Xmx... -jar ...` command for that window. For the MySQL server, use another PowerShell window and the supplied start script; it stays in the foreground. To run any `.ps1` helper from Command Prompt, use:

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\check-laptop.ps1
```

To build Maven from Command Prompt after setting the paths:

```
mvn clean package
```

6. Eclipse setup and run

1. Install an Eclipse IDE for Java with Maven Integration for Eclipse (m2e); keep a Java 17 JDK selected under **Preferences → Java → Installed JREs**.
2. Choose **File → Import → Maven → Existing Maven Projects** (the exact wizard label can differ with installed Eclipse plugins). Browse to the project root `D:\Airline` and select the discovered parent and module POMs. Eclipse's existing-project import instructions cover the root-folder import workflow.
3. If prompted, update Maven project configuration and allow dependencies to resolve. Or right-click the parent project and choose **Maven → Update Project**.
4. Build using **Run As → Maven build...** with goals `clean package`, or run it in Eclipse's integrated terminal after configuring the JDK/Maven paths.
5. Create one **Run Configuration → Java Application** for each class: `DiscoveryApplication`, `AuthApplication`, `FlightApplication`, `BookingApplication`, and `GatewayApplication`. Use **Project root `D:\Airline` as the Working Directory** for all five, especially Auth/Flight/Booking, because `file:./keys/...` resolves relative to the process working folder.
6. Start MySQL, run Discovery first, then Auth / Flight / Booking; wait for their Eureka registration; start Gateway last. Keep each app's Eclipse console running.
7. Build the Angular app in Eclipse's terminal or a separate PowerShell window. Browse `http://localhost:8080` for the packaged app or `http://localhost:4200` while `npm start` is running.

If Eclipse says it cannot find `keys/public.pem` or `keys/private.pem`, first verify that the files exist and that the launch configuration's Working Directory is exactly `D:\Airline`.

7. VS Code setup and run

1. Install the Java Extension Pack and, optionally, Spring Boot Extension Pack.
2. Choose **File → Open Folder...** and open `D:\Airline` (the folder containing the root `pom.xml`).

3. Open an integrated PowerShell terminal, then run:

```
. .\scripts\use-local-tools.ps1
mvn clean package
```

4. Use separate integrated terminal tabs for MySQL, Discovery, Auth, Flight, Booking, and Gateway. Run from the project root so relative key paths resolve.
5. The Java extensions can also launch the `*Application.java` main classes. If so, set their working directory to the project root and use one launch configuration per service.
6. Open a terminal in `airline-frontend`, run `npm ci`, then `npm start` for live UI editing; alternatively run `scripts/package-demo.ps1` for the Gateway-hosted presentation build.

8. Test it from top to bottom

Automated build and unit tests

```
# With Java 17 / Maven available, at the root
mvn test
mvn clean package

# Frontend, at airline-frontend
npm ci
npm run build
```

The current backend suite has six unit tests across Auth, Flight, and Booking. They cover safe customer projection/profile updates, invalid flight rules, seat ordering, and admin report filters. The live acceptance scenario below checks database-backed integration and the UI.

Live acceptance scenario

1. Confirm MySQL listens on 3306 and all five JVM processes are running.
2. Confirm `http://localhost:8761` lists Auth, Flight, Booking as UP.
3. Open `http://localhost:8080`; expect eight flights on a fresh database.
4. Open `http://localhost:8080/swagger-ui/index.html`; choose Authentication, Flights, or Bookings.
5. Customer: log in with the customer demo account; search flights; update the name on **My Profile**; choose a seat; review and confirm; verify the reservation in **My Bookings**; cancel it and verify the seat is available.
6. Admin: log out, then log in with the admin demo account; use **Flights** to review or add a flight and test close/reopen sales with a modal; use **Customers** to search; use **Bookings** to filter by flight/status.
7. Try invalid inputs: blank/too-long profile name, missing login fields, seat 0/over capacity, invalid flight date, non-positive admin flight ID. Expect visible validation and 400 responses from the API.
8. Try role restrictions: a customer cannot open admin pages or admin API routes; an anonymous user cannot open profile or bookings. Expect UI redirect or 401/403.
9. Check a narrow browser width. Tables scroll horizontally; the confirmation modal moves to a bottom sheet; form grids stack into one column.

Useful public URLs:

URL	Expected
`http://localhost:8080`	Angular application (packaged mode).
`http://localhost:8080/#/flights`	Flight search.
`http://localhost:8080/swagger-ui/index.html`	Combined Swagger UI.
`http://localhost:8761`	Eureka dashboard.
`http://localhost:4200`	Angular development server (only while `npm start` runs).

Demo accounts and starter data

Role	Email	Password
Admin	`admin@airline.test`	`Admin@12345`
Customer	`customer@airline.test`	`Customer@12345`

The Auth service inserts each account only if absent; its password is BCrypt-hashed. The Flight service inserts eight sample flights when its table is empty. Sample departure times are set relative to the first fresh database initialization and persist across restarts; they do not roll forward. If a copied database is old, add future flights from Admin before presenting.

9. Understanding `keys/` and `scripts/`

What `keys/` does

`private.pem` lets Auth sign RS256 JWTs. `public.pem` lets Auth, Flight, and Booking verify those signatures and confirm the token has not been altered. The API cannot run protected operations without the matching keys. A fresh clone has no keys because `/keys/` is ignored; generate them once. Do not commit or publicly share the private key. For a classroom machine, both files simply stay beside the project root.

What `scripts/` does

Scripts automate installation and routine commands; they are **not** imported by the Java/Angular runtime. If you install Java/Maven/Node/MySQL yourself and start the programs manually, you can run the application without most scripts. `package-demo.ps1` is a convenience for the Gateway-hosted Angular build, while `start-local-mysql.ps1` is a convenience for running the portable MySQL server. The database and RSA keys remain runtime requirements regardless of whether scripts are used.

Script	Needed every time?	Purpose
`check-laptop.ps1`	No	Reports whether tool versions and MySQL are ready.
`setup-local-tools.ps1`	Once per machine, if using portable tools	Downloads local JDK, Maven, Node/npm and verifies checksums.
`use-local-tools.ps1`	Once per terminal	Selects those tools in the current PowerShell process.
`setup-local-mysql.ps1`	Once per machine, if using portable MySQL	Downloads and initializes local MySQL Server.
`start-local-mysql.ps1`	Each time you use portable MySQL	Runs `mysqld` in the foreground.
`package-demo.ps1`	Each time you want a fresh bundled presentation build	Builds Angular static assets and five JARs.

10. Common problems

Symptom	Check / fix
`java` or `mvn` is not recognized	Dot-source `scripts/use-local-tools.ps1` in PowerShell, or set `JAVA_HOME` / `MAVEN_HOME` and PATH in Command Prompt.
Angular uses a different global CLI	Run from `airline-frontend` and use `npx ng version`. Run `npm ci` first.
`npm ci` shows `EBUSY`	Stop `npm start` and retry the clean install; Windows can temporarily lock a package folder while replacing it.
MySQL connection refused	Confirm **MySQL Server**, not only Workbench, is running on port 3306. Check the local username/password and `DB_PASSWORD`.
`Unknown database airline_*`	Run `database/create-databases.sql` in Workbench or grant permission for the JDBC URL to create schemas.
JPA says a table/column is missing	Check the service startup logs for Flyway migrations and confirm the service connects to its own schema.
`keys/*.pem` not found	Generate keys from the repository root; in Eclipse/VS Code launch configurations, set working directory to root.
Gateway says no instance is available	Wait until the three business services appear UP in Eureka; start Gateway after them. The local registry refresh interval is five seconds.
Website is blank on port 8080	Re-run `scripts/package-demo.ps1` so Angular's latest browser assets are copied into Gateway before rebuilding it.
There are no future flights	Seed dates are based on first database creation. Add a new future flight in Admin or create a fresh classroom database.
Port already in use	Find and stop the old process on 3306, 4200, 8080-8083, or 8761; do not start a second service on the same port.

11. Known toolchain security limitation

On 3 November 2026, `npm audit --omit=dev` reports **6 advisories in the pinned Angular runtime packages (4 high, 2 moderate)**. A full `npm ci` audit reports **32 total** including build-tool dependencies (2 low, 19 moderate, 9 high, 2 critical). This project uses the final Angular 19.2 line to match Node 22.9.0 and CLI 19.2.27; Angular 19 is out of support. The app built and ran successfully for local class use, but keep it local: do not deploy it or expose its demo accounts, root database user, or local HTTP API to the internet. A maintained deployment should upgrade Node and Angular together and repeat the tests.

Official references

- Angular 19.2 compatibility and support status
- Eclipse import an existing project
- MySQL Workbench TCP/IP connections
- MySQL Workbench FAQ and schema creation
- Spring Cloud Netflix Eureka configuration properties